

AstroComms

AstroComms v1.9 / v2.x

Benutzer-Handbuch v1.0

Benoetigt ShadowMD (in unserer Version), AstroComms Firmware, BetterDuino (aktuelle Version), sowie die Sound-Firmware — alternativ MarcDuino.

Version 1.0 — 2026-04-21

Projekt-Repository:

<https://github.com/PrintedDroid/ShadowMD-AstroComms-v1.-v2.x>

AstroComms Ultra Shield — entworfen von walex und nitewing, 2020 (printed-droid.com)

Haftungsausschluss

AstroComms ist ein DIY-Projekt. Aufbau, Verkabelung, Konfiguration und Betrieb umfassen Arbeiten, die bei unsachgemaesser Ausfuehrung Sach- oder Personenschaden verursachen koennen.

Mit der Nutzung dieser Dokumentation, Hard- und Software erkennst du an:

- Du bist alleine verantwortlich fuer alle Arbeiten an deinem System.
- Du verfuegst ueber die noetigen Grundkenntnisse oder holst dir qualifizierte Hilfe.
- Du pruefst Spannung, Polung, Sicherungen und Steckverbindungen vor jedem Einschalten.
- Du verwendest nur passende Komponenten.
- Du beachtest die oertlichen Vorschriften.

Der Autor und alle Beteiligten uebernehmen keinerlei Haftung fuer Schaeden durch falsche Verkabelung, ungeeignete Komponenten, unsachgemaesse Bedienung oder unzureichende Sicherheitsmassnahmen.

Die Dokumentation und Software werden **"wie besehen"** ohne jede Garantie bereitgestellt. Im Zweifel qualifizierte Hilfe einholen bevor am System gearbeitet wird.

Inhaltsverzeichnis

Haftungsausschluss	2
Inhaltsverzeichnis	3
Inhalt	4
1. Was ist AstroComms	4
2. Was du brauchst	4
3. Erste Inbetriebnahme	5
4. Firmware flashen	11
5. SD-Karte vorbereiten	13
6. PS3 Controller einlernen	14
7. Fahrbetrieb	15
8. App per WiFi nutzen	16
9. Referenz der Kommandos	16
10. Motor-Drehrichtung anpassen	17
11. Fehlersuche	18
12. Sicherheit	21

Projekt-Repository (Sketches, **Firmware-Updates,** **Issues**):
<<https://github.com/PrintedDroid/ShadowMD-AstroComms-v1.-v2.x>>

Vollstaendige Benutzeranleitung fuer AstroComms v1.9 / v2.x. Benoetigt ShadowMD (in unserer Version), AstroComms Firmware, BetterDuino (aktuelle Version), sowie die Sound-Firmware — alternativ MarcDuino.

Inhalt

1. Was ist AstroComms
 2. Was du brauchst
 3. Erste Inbetriebnahme
 4. Firmware flashen
 5. SD-Karte vorbereiten
 6. PS3 Controller einlernen
 7. Fahrbetrieb
 8. App per WiFi nutzen
 9. Referenz der Kommandos
 10. Motor-Drehrichtung anpassen
 11. Fehlersuche
 12. Sicherheit
-

1. Was ist AstroComms

AstroComms ist die zentrale Steuerungsplatine fuer einen Astromech-Droiden (R2-D2 oder Variante). Sie traegt alle Controller, die dein Droid fuer Fahren, Dome-Drehung, Panel-Animationen, Sounds und App-Fernsteuerung braucht.

Was du damit machen kannst:

- Fahren und Dome drehen per PS3 Navigation Controller (Bluetooth)
- Fahren und Steuern per Smartphone-App (WiFi)
- Sound-Effekte und Musik abspielen (MP3-Dateien von SD-Karte)
- Dome- und Body-Panels oeffnen, Holos bewegen, Magic Panel animieren
- Bis zu zwei PS3-Controller gleichzeitig (Fuss + Dome) nutzen

Dieses Handbuch richtet sich an die aktuelle **v2.x**-Revision. Falls du noch eine **v1.9** einsetzt, sind die Unterschiede im Abschnitt "Motoren anschliessen" separat dokumentiert.

2. Was du brauchst

Auf der AstroComms-Platine bestueckt (Sockel):

- Arduino Mega 2560 mit **AstroComms Firmware**
- Arduino Mega 2560 mit dieser **ShadowMD-Version** — dieser hat das USB Host Shield darunter sitzen
- ESP32, ESP8266 oder ESP32-C3 Mini — eines der drei Module

- Arduino Nano als **Body Master** mit BetterDuino-Firmware (empfohlen) — MarcDuino als Notloesung moeglich, aber BetterDuino ist deutlich besser gepflegt
- Arduino Pro Mini als **Sound-Controller** — mit DFPlayer_Sound_control_v1.2 (empfohlen) oder MarcDuino V3 im DFPlayer-Mode (Alternative)
- DFPlayer Mini Sound-Modul (auf der Platine gesockelt)

Was du selbst besorgst:

- **Bluetooth-Dongle** — CSR8510-basiert. Empfohlen: LogiLink BT0015 (BT 4.0) oder Asus USB-BT500 (BT 5.0). Funktioniert auch: UGREEN CM748 (BT 5.3), LogiLink BT0048 (BT 4.0), generische CSR8510-Dongles. **Nicht kompatibel:** TP-Link UB400 V2, LogiLink BT0058, die meisten Realtek-basierten BT-5.0-Dongles.
- **PS3 Navigation Controller** — 1 fuer Fahren, optional ein zweiter fuer Dome
- **MicroSD-Karte** fuer den DFPlayer (bis 32 GB, FAT32)
- **Lautsprecher** (3–8 W) und Verstaerker
- **Motortreiber** je nach Aufbau: Sabertooth + SyRen (Serial), VESC/BLDC-ESC (PPM), oder H-Bruecke (PWM+DIR, z.B. Cytron MD oder BTS7960)
- **Netzteil 12 V oder 5 V** mit ausreichender Kapazitaet. **5 V empfohlen. Niemals beides gleichzeitig** einspeisen.

Auf deinem Rechner:

- **Arduino IDE** 1.8.19 oder 2.x (beide getestet) — <https://www.arduino.cc/en/software>
- **xLoader** (Windows) — zum Flashen der .hex -Datei fuer die AstroComms Firmware
- USB-Kabel fuer die Megs, USB-Serial-Adapter (FTDI oder CH340) fuer den Pro Mini
- **VESC Tool** — nur fuer **Mode 1** (Brushless- / VESC-Antrieb). Die offizielle VESC-Konfigurationssoftware. **Pflicht:** jeder VESC/FSESC muss damit einmalig konfiguriert werden, sonst reagiert er nicht auf den Droiden (siehe Abschnitt 3.2).

3. Erste Inbetriebnahme

3.1 Module in die Sockel stecken

Alle Module **stromlos** einstecken. Auf korrekte Orientierung achten (Pin 1 am Silkscreen-Dreieck).

- ShadowMD-Mega mit USB Host Shield unten — in den dafuer vorgesehenen Sockel
- AstroComms-Mega in seinen Sockel
- ESP-Modul — ESP32, ESP8266 oder ESP32-C3 Mini, je eines auf dem 2×4-Sockel
- Nano (Body Master) — in den Nano-Sockel neben den Servo-Anschluessen
- Pro Mini (Sound) — in seinen Sockel
- DFPlayer Mini — in den DFPlayer-Sockel (mit bestueckter SD-Karte)
- Optional: Bluetooth-Dongle ins USB Host Shield

3.2 Motoren anschliessen

Die Motor-Anschlusse unterscheiden sich je nach Platinen-Revision.

Motor-Modi auf einen Blick

AstroComms unterstützt drei Fuss-Antriebs-Modi. **Wähle den Modus zuerst nach deinem Motortyp**, dann nach dem vorhandenen Controller. Die Modus-Namen beziehen sich auf das *Steuersignal*, nicht auf den Motortyp:

Modus	Motortyp	Steuersignal	Beispiel-Controller	Signale pro Motor
Mode 0	Brushed DC	Packetized Serial	Sabertooth 2x32 / 2x25	gemeinsamer Serial-Bus
Mode 1	Brushless (BLDC)	PPM / Servo	VESC, FlipSky FSESC 6.7, Standard-RC-ESC	1 Signaldraht
Mode 2	Brushed DC	PWM + Richtung	H-Brücke: BTS7960, Cytron MD13S	2 Drahte (+ gemeinsame Bremsen)

Die entscheidende Aufteilung:

- **Brushless-Motor** (3 Phasen) → braucht einen ESC/VESC → **Mode 1**
- **Brushed-Motor** (2 Drahte) → Sabertooth seriell (**Mode 0**) oder H-Brücke wie BTS7960/Cytron (**Mode 2**)

■ **Nur Mode 1 treibt Brushless-Motoren. Mode 2 ist ebenfalls brushed** — eine H-Brücke kann keinen 3-phasigen Brushless-Motor kommutieren, trotz der "BLDC-Header"-Beschriftung auf der Platine. Der Modus wird im Sketch ueber `#define FOOT_CONTROLLER` beim Flashen gesetzt (siehe Abschnitt 4).

■ **Mode 1 erfordert eine einmalige VESC-Tool-Konfiguration.** Ein VESC/FSESC reagiert **nicht** out-of-the-box auf den Droiden — jeder Regler muss zuerst mit der VESC-Tool-PC-Software konfiguriert werden (App = PPM, Pulse 1.0–2.0 ms usw.). Ohne das bleiben die Motoren tot, selbst wenn alles korrekt verdrahtet ist. Die Schritt-fuer-Schritt-Anleitung steht im FSESC-Beispiel weiter unten.

Shield v2 (aktuell)

5-Pin-Schraubterminal fuer BLDC- / PWM-Motortreiber:

Silkscreen-Label	Funktion
PWM Left	PWM-Geschwindigkeit linker Fuss-Motor
DIR Left	Drehrichtung linker Fuss-Motor
BRK	Gemeinsame Bremsen (beide Motoren)
PWM Right	PWM-Geschwindigkeit rechter Fuss-Motor
DIR Right	Drehrichtung rechter Fuss-Motor

Plus 2-Pin-Header mit den Signalen PWM Left und PWM Right — nuetzlich fuer PPM/Servo-ESCs wie VESC, die nur eine Signalleitung pro Motor brauchen.

Die Brake-Leitung wird nur in Mode 2 (PWM+DIR) aktiv genutzt. In Mode 1 (PPM) bleibt sie unbenutzt.

Shield v1.9 (aeltere Ausfuehrung)

Der 8-Pin-Pinheader fuehrt die Mega-Pins in dieser Reihenfolge heraus: D19, D17, D32, D11, D8, D7, D6, D5. Alle 8 Pins sind frei nutzbar — sie sind **nicht** fest an die seriellen Busse verdrahtet. D19 / D17 / D32 sind nur Digital (geeignet fuer DIR / Brake), die Pins D5–D11 bieten zusaetzlich PWM.

Silkscreen-Hinweis: D19 und D17 sind auf der v1.9 als RX1 und RX2 beschriftet. Sie waren urspruenglich als Eingaenge fuer zwei Sbus / iBus RC-Empfaenger gedacht (z.B. fuer eine HOTRC DS600 Funkfernsteuerung). Die Shadow-Firmware implementiert diese Option aktuell nicht — die Pins koennen deshalb alternativ als freie Digital-I/O fuer BLDC DIR oder Brake verwendet werden.

Empfohlene Belegung (entspricht dem Sketch-Preset `ASTROCOMMS_SHIELD_V19`):

Funktion	Verwendeter Pin
PWM Left	D6
DIR Left	D32
PWM Right	D7
DIR Right	D8
BRK	D5

D11 bleibt als Reserve fuer zukuenftige Erweiterungen.

Brushless / Mode 1 nutzt auf v1.9 nur D6 und D7. Die Tabelle oben ist die vollstaendige Mode-2-Belegung (brushed, PWM+DIR). Fuer einen Brushless-Antrieb (VESC/FSESC) tragen die beiden PWM-Pins D6 / D7 das PPM-Signal — ein Draht pro Motor — und DIR (D32 / D8) sowie BRK (D5) bleiben unbenutzt.

Zusaetzlich hat v1.9 ein **3-Pin-Cytron-Terminal** (Labels GND / DIR / PWM). Belegt die freien Mega-Pins D3 und D4. Urspruenglich als Alternative zum SyRen10 fuer den Dome-Motor gedacht. Die Shadow-Firmware unterstuetzt diesen Modus aktuell nicht — das Terminal bleibt vorerst unbenutzt.

Mode 0 (Sabertooth-Serial)

Nutzt keinen der BLDC-Pins. Anschluss erfolgt ueber das 6-Pin-Kommunikations-Terminal (siehe 3.3) an den S1-Eingang der Sabertooth. Sabertooth und SyRen teilen sich die Leitung und werden per Packetized-Serial-Adresse unterschieden (128 / 129).

Beispiel: 2× FlipSky FSESC 6.7 (oder VESC) in Mode 1

Wenn du einen BLDC-Antrieb mit zwei VESC-basierten Reglern (FSESC 6.7, BenjaminFB VESC, Trampa VESC o.ae.) aufbaust, brauchst du **Mode 1 (PPM/Servo)**. Jeder Motor bekommt **einen** Signaldraht. Bremse und Richtung kommen NICHT vom Shield — die macht der VESC intern.

Sketch-Konfiguration:

```
// Shield-Revision passend zum Board setzen – GENAU eine aktivieren
// (bleiben beide aktiv, bricht der Compile mit #error ab):
#define ASTROCOMMS_SHIELD_V2      // aktuelle Platine
// #define ASTROCOMMS_SHIELD_V19  // aeltere Platine (stattdessen diese aktivieren)

// Mode 1 = PPM/VESC:
#define FOOT_CONTROLLER 1
```

Pin-Belegung (automatisch durch die Shield-Revision):

Signal	Shield v2	Shield v1.9
leftFootPin	D44 (PWM Left auf 5-Pin-Terminal oder 2-Pin-Header)	D6 (auf 8-Pin-BLDC-Header)
rightFootPin	D45 (PWM Right)	D7 (auf 8-Pin-BLDC-Header)

Verdrahtung pro FSESC (beide Regler identisch):

FSESC-Pin	Zum Shield
Signal (Signalleitung, meist weiss oder gelb)	leftFootPin bzw. rightFootPin
GND (schwarz)	GND-Pin am 6-Pin-Kommunikations-Terminal (oder einer anderen GND-Stelle am Shield)
+5V (rot)	NICHT anschliessen — der FSESC hat eigene Versorgung

Die Stromversorgung des FSESC kommt **separat** vom Antriebsakku an den FSESC. Nur GND ist mit dem Shield gemeinsam.

VESC-Tool-Konfiguration — PFLICHT (pro FSESC einmalig). Ohne sie ignoriert der VESC das PPM-Signal vom Shield und der Droide bewegt sich nicht, egal wie verdrahtet:

1. Motor-Setup-Wizard durchlaufen (Induktanz/Widerstand, Hall oder sensorlos).
2. App Settings → General → App to Use: **PPM** (nicht "PPM and UART", nicht "Nunchuk").
3. App Settings → PPM: - Control Type: **Current** — das ist der richtige Modus fuer einen Droiden, weil: - **Reverse ist aktiv**: Stick nach hinten (<1500 µs) = Motor dreht rueckwaerts. - **Brake ist aktiv**: wenn der Stick in die der aktuellen Fahrtrichtung entgegengesetzte Richtung gedrueckt wird, bremst der VESC aktiv mit negativem Strom (regenerativ, Energie geht zurueck in den Akku). Auch in der Stick-Mitte (1500 µs) bremst der FSESC den freilaufenden Motor. - Alternativen wie Current No Reverse oder Current No Reverse With Brake schalten Teile davon ab — fuer einen Droiden, der sowohl rueckwaerts fahren als auch bremsen koennen soll, ist **nur Current die richtige Wahl**. - Pulse Length Start: 1.0 ms - Pulse Length End: 2.0 ms - Pulse Length Center: 1.5 ms - Deadband: 15–20 % - **Safe Start aktivieren** — verhindert Motor-Anlaufen beim Reconnect in der Mitte der Fahrt
4. Motor Settings → Current → **Motor Current Max Brake** setzen (z.B. 30–60 A negativ, je nach FSESC-Strombudget). Ohne dieses Limit ist die Bremse im Control Type "Current" theoretisch unbegrenzt — realistische Werte machen das Bremsverhalten kontrolliert und schuetzen den Akku.
5. "Write Configuration" und "Write to Flash".

Drehrichtung anpassen: ueber die Shadow-CLI mit `invert left on / invert right on`, dann `invert save`. Ueberlebt den Neustart. Siehe Abschnitt 10.

Nicht verwendete BLDC-Pins: alle anderen BLDC-Header-Pins bleiben offen (v1.9: D5, D8, D11, D17, D19, D32; v2: 42, 43, 40 vom 5-Pin-Terminal). Schaden tut das nichts.

Dome-Antrieb: bleibt unabhaengig. SyRen 10 auf Serial2 treibt den Dome weiter, egal welcher Fuss-Modus gewaehlt ist.

■ ■ FSESCs mit hoher Strom-Einstellung beschleunigen deutlich aggressiver als Sabertooth. **Dry-Test zwingend erforderlich**, bevor der Droide das erste Mal auf den Boden kommt — siehe Abschnitt 12.

3.3 Aeussere Anschuesse

Servo-Leiste neben dem Nano

12 Servo-Anschuesse nummeriert **1 bis 12**. Verbindet Panels / Arme / Tueren am Droiden mit der BetterDuino-Firmware.

Eigenes Servo-Netzteil ueber das separate Spannungs-Terminal. Servos nicht von der System-5V versorgen — hohe Strompeaks beim gleichzeitigen Bewegen stoeren die Elektronik.

Welches Servo an welche Nummer kommt, haengt von der BetterDuino-Rolle des Nano ab:

Body Master (Standard auf dieser Platine):

Servo #	Funktion
1	Data Panel Left (DPL)
2	Utility Arm Upper
3	Utility Arm Lower
4	Left Body Door
5	Left Arm
6	Left Arm Tool
7	Right Body Door
8	Right Arm
9	Right Arm Tool
10	Charge Bay Door
11	in Arbeit
12	in Arbeit

Dome Master (falls der Nano als Dome Master laeuft): Servo 1–10 sind die Dome-Panels 1–10, Servo 11 und 12 sind in Arbeit.

Dome Slave (Holoprojektoren): Servo 1/2 = Front Vertikal/Horizontal, Servo 3/4 = Rear V/H, Servo 5/6 = Top V/H, Servo 7–9 = HP-LEDs (Front/Rear/Top), Servo 10 = zusaetzlicher Kanal, Servo 11 und 12 in Arbeit.

Servo 11 und 12 werden aktuell noch entwickelt — erstmal keine Servos dort anschliessen.

Audio Line Out

Rechts vom Nano: **3,5 mm Klinkenbuchse** und darunter **3-Pin-Header** — beides **dasselbe Line-Out-Signal** vom DFPlayer. An einen externen Verstaerker anschliessen. Nur eines von beiden braucht belegt zu werden (zwei physische Steckoptionen).

Speaker-Test-Terminal (2-Pin)

Fuer einen passiven Lautsprecher direkt am DFPlayer — **nur zum Funktionstest**. Im fertigen Droiden sollte der Sound ueber Line Out in einen Verstaerker gehen.

6-Pin-Kommunikations-Terminal (unten links)

Pin	Signal	Bedeutung
1	GND	Masse
2	Motors	Sabertooth / SyRen Serial-Bus (beide am selben Pin, Adresse 128 bzw. 129)
3	Aux	Optionaler Signalkanal (aktuell keine Funktion, fuer zukuenftige Erweiterungen)
4	MP3	MP3-Command-Signal fuer externe Sound-Ausgabe
5	Flthy TX	FLTHY-HP-Controller-Kommandos
6	Dome TX	Dome-MarcDuino / Dome-BetterDuino-Kommandos

RJ45 zum Dome / Slipring

Fuehrt Aux, MP3, Flthy TX und Dome TX via Slipring-Board in den Dome. Standard-RJ45-Verkabelung.

Slave-Header (rechts neben dem Nano)

Optionaler Anschluss fuer einen zweiten Nano mit BetterDuino in Slave-Rolle — z.B. fuer zusaetzliche Servos oder Panel-Erweiterungen.

RC IN (Pinheader)

Aktuell ungenutzt. Vorgesehen fuer zukuenftige RC-Empfaenger-Integration.

Hinweis: auf der **v1.9** sind die Pins RX1 / RX2 am BLDC-Pinheader (D19 / D17) als alternativer Eingang fuer Sbus- / iBus-RC-Empfaenger vorgesehen — z.B. fuer zwei HOTRC-DS600-Empfaenger. Die Shadow-Firmware bietet dafuer aktuell keine Logik, die Moeglichkeit steht aber fuer zukuenftige Erweiterungen offen.

3.4 Jumper und DIP-Schalter

ESP-Spannung (U76 / U77): einen der beiden Jumper setzen, abhaengig vom ESP-Modul.

- U76 = 5 V zum ESP (z.B. ESP32-DevKit)
- U77 = 3.3 V zum ESP (z.B. ESP8266)

ESP RX/TX-Zuordnung (unter dem ESP-Sockel): einige ESP-Varianten haben vertauschte RX/TX-Pinouts. Mit diesen Jumpern kann die UART-Zuordnung korrigiert werden. Wenn der ESP nach dem ersten Flashen nichts empfaengt oder sendet, diese Jumper pruefen.

PWR-Jumper 1: kann theoretisch die Systemspannung mit der Servo-Versorgung verbinden — **nicht empfohlen**. Servos immer separat versorgen.

MP3-Jumper: unterbricht die MP3-Leitung Richtung Dome (ueber RJ45). Hintergrund: im klassischen MarcDuino-System sendet der Dome Master Sound-Kommandos zum Sound-Modul. Auf dieser Platine uebernimmt der Pro Mini im Body die Sound-Erzeugung, der Weg zum Dome wird nicht gebraucht. Jumper in Default-Stellung lassen, ausser bei Hybrid-Setup.

ISP-Jumper: muss **ausgeschaltet** sein, wenn der Pro Mini per ISP geflasht wird (nur noetig bei MarcDuino V3). Die ISP-Pins fuer den Pro Mini sitzen **unter dem DFPlayer-Modul** — das DFPlayer muss zum Zugang vorher entfernt werden.

DIP-Schalter (3-fach):

- Schalter **1 und 2:** unterbrechen die serielle Kommunikation zum Nano. Vor dem Flashen des Nano beide ausschalten, danach wieder einschalten.
- Schalter **3:** schaltet die Status-LEDs an/aus.

4. Firmware flashen

4.1 ShadowMD per Arduino IDE

1. **Arduino IDE** oeffnen.
2. Sketch-Ordner oeffnen: `Shadow_MD_AstroCan_BLDC_Standalone_v2/` → die `.ino` laedt automatisch.
3. Oben im Sketch zwei Dinge einstellen: - **Shield-Revision:** `` `#define ASTROCOMMS_SHIELD_V2` // aktuelle Platine (Default) `//#define ASTROCOMMS_SHIELD_V19` // aeltere Platine ` - **Motor-Modus:** ` `#define FOOT_CONTROLLER 0` ` - 0 = Brushed DC, Sabertooth Serial - 1 = Brushless ESC/VESC, PPM (z.B. FlipSky F5ESC 6.7) - 2` = Brushed DC, PWM+DIR via H-Bruecke (Cytron MD-Serie, BTS7960)
4. **Board:** Tools → Board → Arduino AVR → Arduino Mega or Mega 2560 .
5. **Port:** den COM-Port der ShadowMD-Mega auswaehlen.
6. **Upload** (Strg-U oder Pfeil-Button).

Bei Erfolg: "Done uploading." im unteren Fensterbereich.

■ ■ **WICHTIG — Dry-Test vor der Erstfahrt:** Nach dem ersten Flashen den Droiden **immer aufbocken** (Raeder frei in der Luft) bevor die Motoren bestromt werden. Falsche Motor-Drehrichtung, falsche Pin-Belegung oder unerwartete Ramping-Werte koennen sonst dazu fuehren, dass der Droide unkontrolliert losfaehrt. Erst wenn alles sauber reagiert — Fahrtrichtung, Lenkung, Stopp — darf er abgesetzt werden. Details siehe Sicherheits-Kapitel 12.

4.2 AstroComms Firmware per xLoader

Die AstroComms Firmware wird als fertige `.hex` -Datei geflasht — kein Source-Code noetig.

1. **xLoader** starten.
2. **Hex file:** `AstroCommsV112.hex` auswaehlen.
3. **Device:** `Mega(ATMEGA2560)` .
4. **COM port:** den Port der AstroComms-Mega waehlen (nicht der ShadowMD!).
5. **Baud rate:** `115200` .
6. Auf **Upload** klicken.

xLoader zeigt unten Fortschritt + Erfolg. Dauert 10–30 Sekunden.

4.3 ESP-Modul per Arduino IDE

Nur noetig, wenn die App genutzt werden soll.

1. **Board-Support installieren:** - File → Preferences → Additional Boards Manager URLs : https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_i

index.json - Tools → Board → Boards Manager → "esp32" suchen → installieren.

2. Sketch oeffnen: esp32_esp8266_as_xbee_v2.0/esp32_esp8266_as_xbee_v2.0.ino .

3. Falls gewuenscht: WLAN-SSID und Passwort anpassen (Default: SSID R2D2 Astromech , Passwort C3P0sucks). **Starkes Passwort dringend empfohlen**, wenn der Droid oeffentlich auftritt.

4. Board waehlen (z.B. ESP32 Dev Module oder NodeMCU 1.0 je nach Modul).

5. Upload Speed: 921600 .

6. Port waehlen und Upload.

Wenn du kein ESP nutzen willst: Sockel leer lassen. Der Rest des Systems funktioniert ohne.

4.4 Nano — BetterDuino (vorgeflasht)

Die BetterDuino Firmware V4 (Autor: RealNobser, Upstream: <https://github.com/RealNobser/BetterDuinoFirmwareV4>) ersetzt die alte MarcDuino-Firmware durch einen modernen, aktiv gepflegten C++-Neubau.

Merkmale:

- Eine Firmware-Datei deckt alle Rollen ab: **Dome Master, Dome Slave, Body Master**. Automatische Konfiguration beim ersten Start, manuell per #MD00 / #MD01 / #MD02 umschaltbar.
- Alle Einstellungen im EEPROM: Servo-Min/Max, Panel-Sequenzen, Sound-Backend, Laufzeit-Justage
- Vier Sound-Hardware-Typen werden unterstuetzt: MP3 Trigger, DF Mini Player, R2-D2 Vocalizer, DY-Player
- Dome-Lift-Integration
- Community-Support: Facebook "Printed Droid"-Gruppe, Astromech.net-Forum

Der Nano kommt auf dieser Platine **vorgeflasht mit BetterDuino in Body-Master-Rolle**. Der End-User muss hier nichts tun. Konfiguration erfolgt ueber Serial-Kommandos (siehe Abschnitt 9).

4.5 Pro Mini — Sound-Controller

Zwei Optionen. **Option A empfohlen — deutlich einfacher, keine spezielle Hardware noetig.**

Option A (empfohlen): DFPlayer_Sound_control_v1.2 per FTDI

Arduino-IDE-Workflow, USB-Serial-Adapter (FTDI / CH340) genuegt:

1. USB-Serial-Adapter an den seriellen Header des Pro Mini anschliessen (GND, 5V, TX-Pin des Adapters an RX des Pro Mini, RX-Pin an TX, DTR an Reset).
2. Arduino IDE: Sketch DFPlayer_Sound_control_v1.2/DFPlayer_Sound_control_v1.2.ino oeffnen.
3. Falls noetig: bank_max_sounds[] (Array am Anfang) auf deine tatsaechlich vorhandenen MP3-Dateien anpassen.
4. Board: Arduino Pro or Pro Mini , Prozessor: ATmega328P (5V, 16 MHz) .
5. Port: den COM-Port des USB-Serial-Adapters.
6. Upload.

Option B (Fallback): MarcDuino V3 per ISP

Nur verwenden, wenn eine bestehende MarcDuino-V3-Installation behalten werden soll. Erfordert ISP-Programmer (z.B. USBasp, AVR Dragon, Arduino-as-ISP), avrdude-Kenntnis und Setzen der Fuses.

1. ISP-Jumper auf der Platine **ausschalten**.
2. DFPlayer-Modul entfernen, um an die ISP-Pins des Pro Mini zu kommen (liegen darunter).
3. ISP-Programmer an die ISP-Pins anschliessen.
4. Fuses setzen: L=0xFF , H=0xD7 , E=0xFC .
5. MarcDuino-V3- .hex flashen.
6. ISP abziehen, DFPlayer zurueckstecken, ISP-Jumper wieder einschalten.
7. Per Serial #SM01 senden, um den MarcDuino in den DFPlayer-Mini-Mode zu versetzen.

Vergleich der beiden Optionen:

	Option A (DFPlayer_Sound_control_v1.2)	Option B (MarcDuino V3)
Flashing	Arduino IDE + USB-Serial-Adapter	ISP-Programmer + avrdude
Fuses setzen	nicht noetig	Pflicht (L=FF / H=D7 / E=FC)
Konfiguration	fertig	#SM01 zum Umschalten in DFPlayer-Mode
Sketch editieren	ja (Arduino IDE)	nur C-Source + Makefile
Aufwand bei Nachbesserung	Arduino IDE, Upload	komplette Toolchain

Unterm Strich: Option A hat weniger Aufwand und weniger Fehlerquellen. Option B ist nur sinnvoll, wenn schon eine MarcDuino-Umgebung laeuft und nicht umgestellt werden soll.

5. SD-Karte vorbereiten

Ein einziges Layout fuer **beide** Sound-Optionen (DFPlayer_Sound_control_v1.2 **und** MarcDuino V3 im DFPlayer-Mode):

1. SD-Karte **FAT32** formatieren.
2. Ordner mp3 im Root anlegen.
3. MP3-Dateien in den mp3 -Ordner legen, mit **4-stelliger Nummer am Dateianfang** und beliebigem Suffix nach einem Unterstrich:

```
mp3/
0001_gen-1.mp3
0002_gen-2.mp3
...
0042_chat-17.mp3
...
0151_leia-1.mp3
...
0254_Startsound.mp3
0255_Startsound.mp3
```

Der DFPlayer erkennt Dateien anhand der **ersten vier Ziffern**. Alles nach dem Unterstrich ist freie Beschreibung — beliebig wählbar.

Bank-Konvention (MarcDuino-Standard):

Bank	Dateien	Typische Inhalte
1	0001–0025	Allgemeine Sounds

Bank	Dateien	Typische Inhalte
2	0026–0050	Chat / Beeps
3	0051–0075	Happy
4	0076–0100	Sad
5	0101–0125	Whistles
6	0126–0150	Scream
7	0151–0175	Leia / Nachrichten
8	0176–0200	Musik / Cantina / Themes
9	0201–0225	Reserviert
—	0254, 0255	Startup-Sounds (MarcDuino-Konvention)

Nicht jede Bank muss voll sein. Wenn du mit Option A arbeitest, kannst du im Sketch am Array `bank_max_sounds[]` die tatsächlichen Zahlen anpassen.

Lautstaerke-Default: mittig (ca. 15 von 30). Zur Laufzeit ueber den PS3-Controller oder CLI aenderbar (siehe Abschnitt 9).

6. PS3 Controller einlernen

Das System muss die MAC-Adressen deiner PS3-Controller einmalig lernen. Ablauf via Serieller Konsole.

6.1 Vorbereitung

- Frisch aufgeladene PS3 Navigation Controller
- USB-Kabel zum Controller (mini-USB)
- ShadowMD-Mega eingesteckt und per USB angeschlossen
- Bluetooth-Dongle **noch nicht** stecken

6.2 Pairing-Ablauf

1. Arduino IDE → **Tools** → **Serial Monitor**. Baudrate unten rechts auf **115200** stellen.
2. `pair` tippen und Enter druecken.
3. Die Konsole meldet, welcher Slot gerade dran ist: Fuss → Dome → Fuss-Ersatz → Dome-Ersatz.
4. **Pro Controller:** - BT-Dongle abziehen (falls gesteckt). - PS3-Controller per USB-Kabel direkt am USB-Host-Shield des ShadowMD-Mega anstecken. - Die Konsole meldet "Paired [Slot]: XX:XX:XX:XX:XX:XX". - Controller abziehen. - BT-Dongle wieder einstecken.
5. Bei zwei Controllern den Ablauf ein zweites Mal durchlaufen. Spare-Slots (3 und 4) sind optional.
6. Wenn fertig: `done` tippen und Enter.
7. `status` zeigt die gespeicherten MACs zur Kontrolle.

Bei Fehlversuch: `clear` loescht alle Eintraege, dann von vorn mit `pair` .

6.3 Controller verbinden

Nach erfolgreichem Pairing:

1. Mega neu starten (Reset-Button).
2. BT-Dongle einstecken.
3. Am PS3-Controller **PS-Taste** druecken.
4. Nach ca. 5 Sekunden leuchtet eine der vier Controller-LEDs dauerhaft → verbunden.

Der erste Controller ist der Fuss-Controller, der zweite (falls verbunden) uebernimmt den Dome. **Der zweite Controller braucht erfahrungsgemaess laenger zum Verbinden als der erste** — Geduld.

6.4 Controller ausschalten

- **PS-Taste 15 Sekunden halten**, oder
- **PS + L3 gleichzeitig**

7. Fahrbetrieb

PS3 Fuss-Controller

- **Analog-Stick**: vor / zurueck / links / rechts — Fahrt plus Lenkung (BHD Diamond Mixing)
- **L1 + Stick** oder **L2 + Stick** (je nach Config): Dome-Drehung, falls kein zweiter Controller verbunden ist
- **Kreuz-Tasten + Dreieck / Quadrat / Kreis / X**: konfigurierbare Shortcuts fuer MarcDuino-Sequenzen, Sounds, Panels
- **PS-Taste lang**: Controller trennen

PS3 Dome-Controller (optional)

- **Analog-Stick**: Dome-Rotation
- Eigene Tasten-Shortcuts fuer Dome-Panels, Holos, Sounds

Automatische Dome-Bewegungen

Wenn aktiviert, dreht der Dome zufaellig in Ruhephasen. Aktivierung ueber Tasten-Shortcut (im Sketch konfiguriert).

Sound-Shortcuts

Schnell ueber PS3-Tasten ausloesbar:

Shortcut	Sound
\$L	Leia-Nachricht
\$C	Cantina (Orchester)
\$c	Cantina Beep (R2-Version)
\$S	Scream
\$F	Faint / Kurzschluss
\$D	Disco
\$W	Star Wars Theme
\$M	Imperial March

8. App per WiFi nutzen

Wenn der ESP gesteckt und geflasht ist:

1. Droid einschalten.
2. Am Smartphone/Tablet: WLAN R2D2 Astromech auswaehlen, Passwort C3P0sucks (oder das selbst gesetzte).
3. R2-Steuerungs-App oeffnen, Verbindungstyp **TCP** waehlen.
4. **Host:** 192.168.4.1
5. **Port:** 9750
6. Verbinden.

Bis zu **4 Geraete** gleichzeitig moeglich (z.B. dein Phone + Tablet + ein Button-Board eines Freundes). Die AstroComms Firmware queueet Befehle FIFO, damit keine Kollisionen entstehen.

Die App kann alle Standard-MarcDuino-Kommandos senden.

XBee-Randbemerkung: der ESP-Sockel kann theoretisch auch mit einem klassischen XBee-Modul statt eines ESP bestueckt werden. In der Praxis wird fuer diese Platine der ESP-Weg verwendet — einfacher, schneller, unverschluesst bei normalem Betrieb, und alle aktuellen R2-Apps unterstuetzen TCP.

9. Referenz der Kommandos

9.1 ShadowMD CLI (Serial Monitor, 115200 Baud, am ShadowMD-Mega)

Kommando	Bedeutung
help	Liste aller Befehle
status	Aktueller Status: gepaarte Controller, Verbindung, Invert-Settings
pair	PS3-Pairing-Modus starten
done	Pairing-Modus beenden
clear	Alle gepaarten Controller loeschen
invert	Motor-Drehrichtungs-Einstellung anzeigen
invert left on / off	Linke Seite invertieren oder normal
invert right on / off	Rechte Seite invertieren oder normal
invert save	Invert-Einstellung dauerhaft speichern
invert reset	Beide Seiten auf normal setzen und speichern
debug	Verbose Bluetooth-Debug-Ausgabe an/aus
reboot	Software-Reset

9.2 Sound- und Panel-Kommandos (ueber App / WiFi / XBee)

Alle Kommandos enden mit Carriage Return (\r).

Praefixe:

- `:` = Dome-Kommando (z.B. `:SE01` = Dome-Panel-Sequenz 01)
- `;` = Body-Kommando
- `$` = Sound
- `+` = FLTHY-HP direkt
- `*` = Broadcast Dome + FLTHY
- `_` = Body-Config

Haeufige Kommandos:

Kommando	Wirkung
<code>:SE01</code>	Dome-Sequenz 1 (Close All Panels)
<code>:OP00</code>	alle Dome-Panels oeffnen
<code>\$R</code>	Random-Sound-Modus an
<code>\$0</code>	Mute
<code>\$s</code>	Sound stoppen
<code>\$+ / \$-</code>	Volume hoch / runter
<code>\$m / \$f / \$p</code>	Volume Mitte / Max / Min
<code>\$L , \$C , \$c , \$S , \$F , \$D , \$W , \$M</code>	Sound-Shortcuts (siehe Tabelle oben)

9.3 MarcDuino-V3-Config-Kommandos (nur bei Option B)

Am Pro Mini Serial Monitor (9600 Baud), bzw. ueber den Body-Bus:

Kommando	Wirkung
<code>#SM00</code>	Sound-Backend MP3 Trigger (Default)
<code>#SM01</code>	Sound-Backend DF Mini Player
<code>#SQ00 / #SQ01</code>	Chatty-Modus an / aus
<code>#SS00 – #SS03</code>	Startup-Sound aus / Default / Alt 1 / Alt 2
<code>#SD00 / #SD01</code>	Alle Servo-Richtungen normal / reversed
<code>#SRxxy</code>	Einzelne Servo-Richtung setzen (xx=Nr, y=0/1)

10. Motor-Drehrichtung anpassen

■ ■ **Zuerst aufbocken!** Bevor du Invert-Einstellungen testest, den Droiden aufbocken (Raeder frei). Wenn `invertLeft` verkehrt herum ist, dreht der linke Motor bei Vorwaerts-Stick rueckwaerts — der Droide zieht unkontrolliert seitwaerts weg. Erst wenn die Richtung sauber stimmt, wieder auf den Boden stellen.

Wenn die Fahrtrichtung beim ersten Test verkehrt ist (z.B. linker Motor dreht falsch):

1. Serial Monitor am ShadowMD-Mega oeffnen (115200 Baud).
2. `invert left on` → Enter. Testen.
3. Wenn rechts auch verkehrt: `invert right on` .
4. Wenn alles stimmt: `invert save` → Einstellung ueberlebt den Neustart.

Rueckstellung: `invert reset` .

Hinweis: wirkt nur bei BLDC-Modi (FOOT_CONTROLLER 1 und 2). Bei Sabertooth-Mode gibt es eine separate Invertierung, die im Sketch-Code geaendert wird.

11. Fehlersuche

Sketch laesst sich nicht kompilieren

- `#error "No shield revision defined"` : eine der beiden Shield-Zeilen am Sketch-Anfang muss aktiv sein (`ASTROCOMMS_SHIELD_V2` oder `ASTROCOMMS_SHIELD_V19`).
- `#error "Define only ONE shield revision"` : beide Zeilen aktiv — eine auskommentieren.
- **USB-Host-Shield-Library-Konflikt:** der Ordner `src/` im Sketch-Verzeichnis muss erhalten bleiben. Nicht durch eine Arduino-Library-Manager-Version ueberschreiben.

Controller verbindet nicht

- BT-Dongle kompatibel? LogiLink BT0015 und Asus USB-BT500 sind empfohlen; TP-Link UB400 V2 und Realtek-BT-5.0-Dongles funktionieren typischerweise nicht.
- Pairing durchgefuehrt? `status` im Serial Monitor pruefen — wenn "XX" statt MAC, ist der Slot leer.
- Nach `pair` nicht vergessen `done` zu senden, bevor die Verbindung aufgebaut wird.
- Mega neu starten, danach PS-Taste.
- Zweiter Controller braucht mehr Zeit zum Verbinden als der erste — nicht ungewoehnlich.

Kein Sound

- SD-Karte FAT32? `mp3/` -Ordner vorhanden? Dateien mit 4-stelligem Praefix?
- DFPlayer-Modul korrekt gesockelt?
- Lautsprecher am Line Out (Klinke oder 3-Pin-Header) via Verstaerker, **nicht** am 2-Pin-Speaker-Testterminal im Dauerbetrieb?
- Am Pro Mini einen USB-Serial-Adapter anstecken, Monitor mit 9600 Baud oeffnen, `$101` senden → muss Datei `0001_...mp3` abspielen.
- Wenn der Pro Mini MarcDuino V3 laeuft: `#SM01` senden, um den DFPlayer-Mode zu aktivieren (Default ist MP3-Trigger).

Fahrtrichtung verkehrt

- `invert left on` bzw. `invert right on` , `invert save` — siehe Abschnitt 10.

WiFi nicht sichtbar

- ESP-Modul gesteckt? Spannungs-Jumper richtig gesetzt (U76 = 5 V, U77 = 3.3 V)?
- RX/TX-Jumper unter dem ESP korrekt? Manche ESP-Module haben vertauschte Pinouts.

- ESP per USB an PC anschliessen, Serial Monitor (9600 Baud) → "Access Point started successfully" sichtbar?
- Smartphone im 2.4-GHz-WiFi-Band? ESP32/ESP8266 unterstützen nur 2.4 GHz.

Motor zuckt / ruckelt

- Netzteil zu schwach? Stromspitzen beim Beschleunigen erfordern ordentliche Leistung.
- Lose Schraub-/Steckverbindung am Motor-Terminal oder Motor-Controller? Alle nachziehen.
- BLDC Mode 2 ohne Brake verdrahtet? Brake-Pin anschliessen oder `ENABLE_BRAKE` im Sketch auskommentieren.

App verbindet, aber nichts passiert

- Port wirklich **9750**?
- App-Verbindungstyp = **TCP**, nicht XBee-API.
- Telnet-Test von PC: `telnet 192.168.4.1 9750` — sollte offen sein.
- ESP-Firmware geflasht?

Pro Mini lässt sich nicht flashen (MarcDuino-Weg)

- ISP-Jumper ausgeschaltet?
- DFPlayer-Modul entfernt, um an die ISP-Pins zu kommen?
- Richtige Fuses gesetzt (`L=0xFF / H=0xD7 / E=0xFC`)?
- Alternativ auf Option A (DFPlayer_Sound_control_v1.2) via FTDI umsteigen — deutlich einfacher.

Nano (BetterDuino) lässt sich nicht per Serial ansprechen

- DIP-Schalter 1 und 2 eingeschaltet? (Wenn sie aus sind, ist die Serial-Kommunikation zum Nano getrennt.)

Status-LEDs aus

- DIP-Schalter 3 einschalten.

Servos zucken / sind ueberlastet

- Servo-Netzteil separat angeschlossen, PWR-Jumper 1 in Default-Stellung (nicht ueberbrueckt)?
- Servo-Netzteil liefert genug Strom fuer alle Servos gleichzeitig?

Servo 11 oder 12 bewegen sich nicht

- Diese beiden Kanäle sind aktuell **in Arbeit** — bitte noch keine Servos dort anschliessen.

Dome dreht sich nicht, obwohl Fuss-Motoren funktionieren

- SyRen DIP-Schalter: Packetized-Serial-Mode (1, 2 und 4 Down, alle anderen Up)?
- SyRen-Motor-Anschluss M1/M2 vertauscht?
- Adresse 129 auf der SyRen-Platine korrekt?

Dome dreht unkontrolliert schnell oder ruckartig

- Im Sketch `domespeed` reduzieren (Standard ist 100, Bereich 0-127).
- Ramping-Parameter pruefen — zu aggressives Ramping erzeugt Ruckeln.

Zwei Controller verbinden nicht gleichzeitig

- CSR4-Dongles koennen oft nur **einen** Controller gleichzeitig halten — das ist eine Dongle-Limitation, kein Firmware-Problem.
- Loesung: CSR8510-basierten Dongle verwenden (LogiLink BT0015, Asus USB-BT500, siehe Abschnitt 2).
- Alternativ: zweiten Controller erst nach ~10 s verbinden, nachdem der erste stabil laeuft.

Sound setzt aus / knistert

- Spannungsabfall am DFPlayer — eigene Audio-Versorgung oder Puffer-Kondensator am Modul.
- SD-Karte der Klasse 10 oder hoeher verwenden, bekannte Marken (SanDisk, Samsung) — billige Fake-Karten sind haeufige Ursache.
- SD-Karte neu formatieren (FAT32), Files ohne Fragmentierung neu aufspielen.

Kompilier-Fehler "xxx.h: No such file or directory"

- Gebuendelter `src/` -Ordner im Sketch fehlt oder wurde durch Arduino-Library-Manager ueberschrieben.
- Sketch aus dem Git-Release/Zip neu holen, `src/` -Ordner darf **nicht** entfernt werden.

Nach Firmware-Update: `status` zeigt "XX" statt MAC

- EEPROM-Magic-Byte stimmt nicht mehr — passiert nach `clear` , First-Boot eines neuen Megas oder einer EEPROM-Korruption.
- Einfach `pair` neu durchlaufen, siehe Abschnitt 6.

BT-Dongle wird heiss / Verbindung bricht staendig ab

- Qualitaets-/Versorgungs-Problem des Dongles. Anderen Dongle aus der Empfehlungsliste (Abschnitt 2) testen.
- USB-Kabel kurz halten, HighSpeed-Kabel verwenden.
- Abstand zu stoerenden Quellen (WLAN-Router, Mikrowelle, andere BT-Geraete).

App-Kommando kommt nicht beim Droiden an

- ESP-Firmware wirklich geflasht? Am ESP-USB-Serial-Monitor (9600 Baud) nach "Access Point started successfully" sehen.
- Kein XBee gleichzeitig im Sockel? (ESP und XBee schliessen sich am selben UART gegenseitig aus.)
- App-Verbindungstyp **TCP** (nicht XBee-API-Mode).
- Test: am AstroComms-Mega USB-Debug anschliessen, Serial Monitor (9600 Baud) — dort muesste das Kommando vom ESP eintreffen.

Droide reagiert verzoegert auf Stick-Input

- BT-Interferenz durch nahen WLAN-Router, Mikrowelle oder andere BT-Geraete.
- Dongle mit externer Antenne besser als Mini-Stummel-Dongle.
- Abstand und Sichtlinie zum Droiden halten.

Servo zittert im Ruhezustand

- Viele Billig-Servos oszillieren, wenn sie dauerhaft unter Strom stehen.
- BetterDuino hat einen Servo-Detach-Mechanismus — per Serial-Kommando konfigurierbar (siehe BetterDuino-Repo).
- Bessere Servo-Modelle (Digital, Metallgetriebe) zittern deutlich weniger.

12. Sicherheit

Dry-Test bei Erstinbetriebnahme (WICHTIG)

Vor dem ersten Fahrbetrieb immer Trockenlauf:

- Droiden **aufbocken** (Holzklotz, Kiste, Montageboecke) bis die Raeder frei in der Luft sind.
- ShadowMD-Mega per USB anschliessen, Stromversorgung der Motoren trennen.
- Controller verbinden, Fuss-Stick sanft bewegen → pruefen, **in welcher Richtung** jede Seite dreht.
- Wenn falsch: `invert left on` / `invert right on` / `invert save` (siehe Abschnitt 10).
- Dome-Rotation und Panel-Bewegungen ebenfalls ohne Fahrbetrieb erst im Leerlauf pruefen.
- **Erst wenn Fahrtrichtung, Lenkung, Ramping und Not-Stopp sauber reagieren**, den Droiden absetzen.

Bei BLDC-Setups besonders wichtig — PWM-Kurven, Brake-Verhalten und Beschleunigung sind ohne Last schwer vorhersehbar. Ein Droide der bei der ersten Fahrt ungebremst losschiesst, ist ein reales Verletzungsrisiko.

Not-Stopp

BT-Dongle waehrend des Fahrbetriebs abzuziehen stoppt die Motor-Verbindung (Timeout). Ein **zusaeztlicher Hardware-Not-Aus-Schalter** in der Stromversorgung ist bei Live-Auftritten Pflicht. Der Schalter muss fuer den Fahrer jederzeit erreichbar sein.

Beim Flashen

Vor jedem Firmware-Upload: **Motoren stromlos machen** (Fuss + Dome trennen). Ein halb-hochgeladener Sketch kann ungewollte Signale auf den Motor-Pins erzeugen.

Stromversorgung

12 V **oder** 5 V speisen — **niemals beides gleichzeitig. 5 V empfohlen**. Ueberspannungsschutz und ausreichende Kapazitaet pruefen.

Pairing-Hygiene

Nach Conventions: Spare-Slots (3 und 4) via `clear` zuruecksetzen, damit fremde Controller sich nicht versehentlich verbinden.

WiFi-Passwort

Das Default-Passwort `C3P0sucks` ist **oeffentlich bekannt**. Fuer Events unbedingt aendern (Abschnitt 4.3) — Zugriff auf das WiFi bedeutet vollen Zugriff auf den Droiden.

Separates Servo-Netzteil

Servos **immer separat** versorgen. PWR-Jumper 1 in Default-Stellung lassen. Hohe Stromspitzen beim gleichzeitigen Bewegen mehrerer Servos koennen sonst Reset-Events oder USB-Drops verursachen.

Vor-Einschalt-Pruefung

Vor dem ersten Einschalten nach Umbauten oder neuen Komponenten:

- Polung der Akku-Leitungen mit Multimeter pruefen (Plus an Rot, Minus an Schwarz)
- Keine Kurzschlusse zwischen Power und GND (Widerstandstest im ausgeschalteten Zustand)
- Passende Sicherung in der Hauptleitung — entsprechend dem Kabelquerschnitt und der Motor-Last
- Alle Schraubterminals festgezogen, keine Litzen die sich loesen koennen

Batterie-Sicherheit (LiPo)

- **Balance-Laden** mit geeignetem Ladegeraet, nie aufgeblaehrte oder geschwollene Zellen verwenden.
- **LiPo-Safe-Bag** oder feuerfester Behaelter zum Lagern und Laden.
- **Waehrend Transport abklemmen** — Wackelkontakte koennen beim Transport zu Kurzschluessen werden.
- Nie ueberladen, nie tiefentladen, nie in direkter Sonne lassen.

Gemeinsamer GND bei getrennten Netzteilen

Bei getrennter Versorgung von Motoren und Logik: die **GND-Leitungen beider Netzteile miteinander verbinden**. Ohne gemeinsame Masse driften die Potentiale und Steuersignale koennen kaputt gehen oder unvorhersehbare Schaltungen ausloesen.

Thermik der Motortreiber

Sabertooth, SyRen, BLDC-ESCs und H-Brueckentreiber werden im Fahrbetrieb **warm**. Im Normalbetrieb ist das OK — bei Ueberhitzung reduziert sich die Leistung automatisch.

- Luftzirkulation im Droiden sicherstellen, keine Stoff-/Schaum-Abdeckung direkt am Treiber.
- Bei langen Fahrphasen (Conventions) gelegentlich pausieren und Temperatur pruefen (Handruecken).
- Passive Kuehlkoerper oder kleine Luefter sind bei starken Motoren empfehlenswert.

ESD (elektrostatische Entladung)

Bei trockenem Wetter oder Teppichboden:

- Vor dem Anfassen von MCU-Modulen, ESP, Nano, Pro Mini erst einen geerdeten Metallgegenstand beruehren (Heizungsrohr, Metallgehaeuse).
- Module nie an den Pins anfassen — nur an den Kanten.
- Keine Nylon-/Synthetik-Kleidung beim Umstecken.

Event-/Convention-Sicherheit

Bei oeffentlichen Auftritten:

- **Sichtkontakt** zum Droiden halten — nie "blind" fahren.

- **Sicherheits-Abstand** zu Kindern, Haustieren, Fotografen — der Droide hat keine Sensorik, er weicht nichts aus.
- **Not-Aus griffbereit** fuer den Fahrer.
- **Akku-Reserve** beachten — leere LiPos koennen bei Motor-Last abrupt einbrechen, dann reagiert der Droide unvorhersehbar.
- **Nicht waehrend des Ladens fahren** — das ist wie Rasenmaehen mit offenem Stecker.

Viel Spass mit deinem Droiden!

Weitere Informationen und Community-Support: printed-droid.com, astromech.net, Facebook-Gruppe "Printed Droid".